

EVENTFLOW

EF-DOC-004

Software Architecture

Version 0.1 | Draft for Architecture Review

Sprint 2 - Architecture & Domain Model

Every Guest Matters. Every Event Flows.

Document Control

Document ID	EF-DOC-004
Title	EventFlow Software Architecture
Version	0.1
Status	Draft for Architecture Review
Product	EventFlow
Document Owner	Product Owner
Technical Owner	Solution Architect
Created	2026-08-06
Last Updated	2026-08-06
Classification	Internal Software Architecture
Related Documents	EF-DOC-000, EF-DOC-001, EF-DOC-002, EF-DOC-003, EF-DOC-005

Revision History

Version	Date	Status	Summary
0.1	2026-08-06	Draft for Architecture Review	Initial target architecture derived from approved Sprint 1 governance, ADR, and PRD baselines.

Approval Record

Role	Approval	Date
Product Owner	Pending Review	-
Solution Architect	Draft Prepared	2026-08-06

1. Purpose

This document defines the target software architecture for EventFlow v1.x. It translates the approved product requirements and architecture decisions into a coherent technical blueprint for implementation.

The architecture is intentionally designed to support the current WordPress reference deployment while preventing the product domain from becoming permanently coupled to event-specific pages, external communication providers, or one-event assumptions.

This is a **design document, not implementation approval**. Architectural choices identified as Proposed require review and, where material, an accepted ADR before implementation.

2. Architectural Objectives

The EventFlow architecture shall:

1. Preserve **Event** as the root business context.
2. Keep **Invitation** and **Attendee** as distinct domain concepts.

3. Support configurable event identity, branding, venue, schedule, communication templates, and deadlines.
4. Keep guest-facing workflows secure and mobile-first.
5. Separate domain rules from WordPress page rendering and admin screens.
6. Isolate email/SMS providers behind replaceable adapters.
7. Support controlled schema migrations from the current reference plugin.
8. Scale through Tier A-C reference capacity, up to 5,000 attendees per event, without fundamental domain-model redesign.
9. Support reliable event-day reception operations even if non-critical external integrations are unavailable.
10. Preserve traceability across requirements, ADRs, schema, components, tests, and releases.

3. Architecture Context

3.1 Current Reference State

The current Lui @ 60 reference plugin successfully demonstrates:

- Excel/CSV guest import;
- generated guest IDs;
- high-entropy personalized invitation tokens;
- personalized invitation pages;
- invitation capacity;
- companion-name collection;
- a simple dashboard.

The current implementation is intentionally a prototype/reference foundation. It contains implementation shortcuts that must not become permanent product architecture, including:

- event-specific naming in code;
- a single primary guest table;
- companion names stored as JSON;
- event details embedded in presentation logic;
- incomplete separation of admin UI, domain logic, persistence, and integrations.

3.2 Target State

EventFlow v1.x shall evolve the reference implementation into a **modular application architecture** with explicit domain modules, application services, infrastructure adapters, and controlled persistence.

4. Proposed Architecture Style

4.1 Proposed: Modular Monolith for v1.x

EventFlow v1.x is proposed as a **modular monolith deployed as a WordPress plugin/application module**, rather than as independent microservices.

Rationale:

- the current deployment environment is WordPress;
- operational scale is Tier A-C, not global SaaS scale;
- a single deployable unit reduces operational complexity;
- strong internal module boundaries provide most of the maintainability benefits needed at this stage;
- future extraction of services remains possible if interfaces and domain boundaries are kept explicit.

This proposal requires an ADR before becoming an accepted implementation rule.

4.2 Layered Structure

The target logical layers are:

1. Presentation Layer

- WordPress admin pages
- guest-facing invitation pages
- reception UI
- REST controllers/endpoints

2. Application Layer

- use-case orchestration
- commands/services
- authorization coordination
- transaction boundaries

3. Domain Layer

- Event
- Invitation
- Attendee
- Communication
- Table / Seating Assignment
- Check-in
- business rules and state transitions

4. Infrastructure Layer

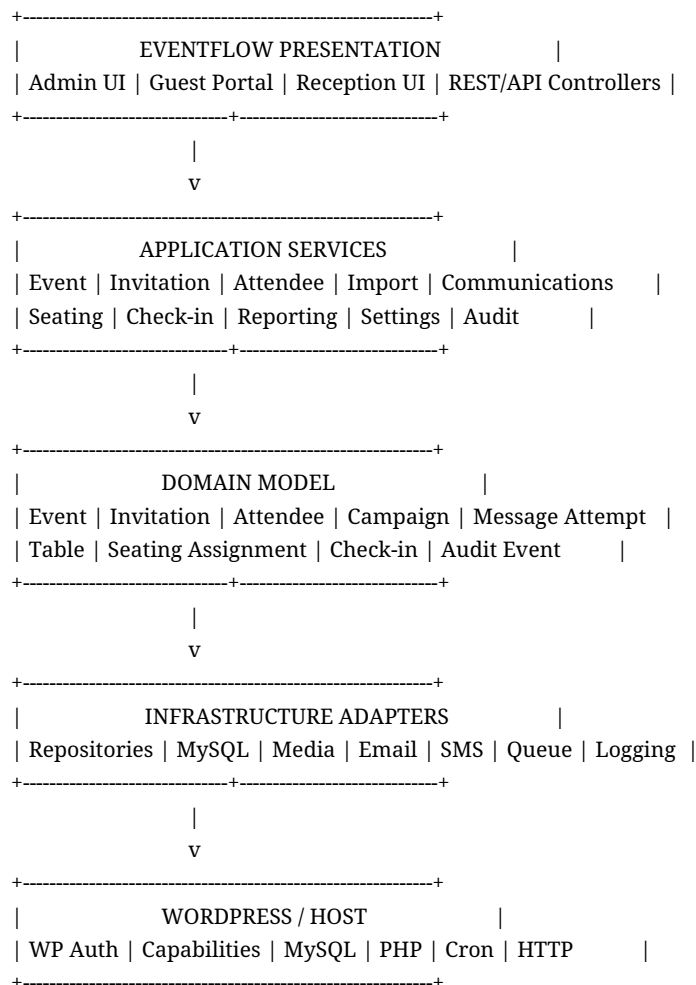
- WordPress/MySQL repositories
- email/SMS provider adapters
- file/media storage

- queue/background-job adapters
- logging
- clock/UUID/token generation

5. Platform Layer

- WordPress core
- authentication/session
- capability system
- cron/background facilities
- MySQL
- web server / PHP runtime

5. High-Level Component Model



6. Domain Modules

6.1 Event Module

Responsibilities:

- event identity and lifecycle;
- event date/time/timezone;
- venue reference and event schedule;
- branding and invitation artwork;
- event-wide wording and deadlines;
- event status;
- event-scoped configuration.

The Event module owns event configuration. Other modules reference Event configuration but do not duplicate it as their source of truth.

6.2 Invitation Module

Responsibilities:

- invitation identity;
- primary invitee/contact;
- invitation capacity;
- secure token lifecycle;
- invitation status;
- confirmation status;
- invitation-level communication context;
- invitation access rules.

Invitation does **not** own seating assignments for individual attendees.

6.3 Attendee Module

Responsibilities:

- person identity within an Event;
- role relative to Invitation (primary/companion/other future roles);
- confirmation state;
- attendee-level operational attributes;
- seating assignment reference;
- check-in state;
- future badge/place-card attributes.

Companion names shall migrate from JSON-only storage into individual Attendee records.

6.4 Communications Module

Responsibilities:

- templates;
- campaigns/audience definitions;
- channel selection;
- message generation;
- batching;
- message attempts;

- provider adapter calls;
- delivery-status updates;
- retry state;
- communication audit/history.

The module shall not change Invitation or Attendee data merely because a provider call succeeds or fails, except for explicitly defined communication-status projections.

6.5 Seating Module

Responsibilities:

- table definitions;
- capacity;
- table status;
- attendee assignments;
- capacity validation;
- later grouping/layout functions.

The visual floor-plan planner is deferred until venue data is known.

6.6 Reception / Check-in Module

Responsibilities:

- fast attendee search;
- table lookup;
- invitation relationship display;
- attendee arrival;
- duplicate check-in handling;
- event-day operational status.

Reception must not require live access to email/SMS providers.

6.7 Import / Export Module

Responsibilities:

- XLSX/CSV ingestion;
- header mapping;
- row validation;
- duplicate detection;
- import staging;
- safe apply/upsert;
- rejection/error report;
- export generation.

Import should use a staged pipeline instead of directly mutating production records row-by-row without review at larger scale.

6.8 Administration / Audit Module

Responsibilities:

- roles and capabilities;
- integration settings;
- system health;
- audit event recording;
- migration status;
- controlled maintenance actions.

7. Application Service Boundaries

Application services coordinate use cases but should avoid becoming generic "god services."

Proposed services include:

- EventService
- InvitationService
- AttendeeService
- InvitationAccessService
- GuestConfirmationService
- ImportService
- CommunicationService
- CampaignService
- SeatingService
- CheckInService
- ReportingService
- SettingsService
- AuditService

Each service should depend on interfaces/repositories, not direct SQL scattered through controllers or templates.

8. Persistence Architecture

8.1 Proposed: Custom EventFlow Tables

EventFlow domain records are proposed to use purpose-built custom MySQL tables rather than storing core operational entities as WordPress posts/postmeta.

Reasons:

- predictable relational structure;
- explicit event ownership;
- efficient attendee and invitation queries;
- indexes designed for event operations;
- cleaner migrations;

- less metadata-table amplification;
- easier reporting and integrity rules.

WordPress options may still store platform-wide configuration that is not event-domain data.

This proposal requires ADR acceptance.

8.2 Repository Pattern

Persistence access should be encapsulated through repositories such as:

- EventRepository
- InvitationRepository
- AttendeeRepository
- CommunicationRepository
- TableRepository
- CheckInRepository
- AuditRepository

Repositories provide domain-oriented operations and hide WordPress \$wpdb details from application services.

8.3 Transaction Boundaries

Operations that modify multiple related records should use explicit transaction boundaries where the hosting database/runtime supports reliable transactions.

Examples:

- converting a guest confirmation into multiple Attendee records;
- replacing an invitation's companion set;
- seating reassignment;
- import apply step.

9. Target Data Ownership

10. Guest Access Architecture

10.1 Token-Based Access

Guest-facing invitation access shall continue to use a high-entropy opaque token.

Public URLs should contain:

`/invite/{opaque-token}/`

and should not contain email addresses, sequential invitation IDs, or personally identifiable information.

10.2 Token Storage

Proposed security improvement:

- store a token identifier/lookup value suitable for resolving requests;
- consider storing only a cryptographic hash of the token at rest if operational requirements permit;
- support token rotation/revocation.

Exact token-at-rest design requires a security ADR before implementation.

10.3 Guest Session

Guest confirmation does not require a WordPress account in v1.x.

A successfully resolved invitation establishes only the minimum invitation context needed for that request.

11. Administrative Authorization

WordPress authentication remains the v1.x administrative identity provider.

EventFlow shall define product capabilities mapped to roles such as:

- EventFlow Administrator
- Event Organizer
- Event Coordinator
- Reception Staff
- Reporting / Read Only

Authorization checks belong at application/controller boundaries, not only in hidden menu items.

12. API and Controller Architecture

12.1 Proposed: WordPress REST API for Structured Interactions

REST endpoints are proposed for:

- admin data grids;
- guest confirmation submission;
- check-in;
- table assignments;
- communication actions;
- future asynchronous UI operations.

REST controllers shall:

- authenticate/authorize as appropriate;
- validate request shape;
- call application services;
- return normalized responses;
- contain minimal business logic.

Public token endpoints require rate limiting/throttling considerations in the security design.

12.2 Server-Rendered Compatibility

Public invitation pages may remain server-rendered in v1.x for simplicity, progressive enhancement, SEO irrelevance, and resilience.

JavaScript should enhance rather than become the sole mechanism for core guest confirmation unless later requirements justify a full SPA.

13. Communications Architecture

13.1 Provider Adapter Interface

A provider adapter should expose an EventFlow-oriented contract rather than leaking provider-specific APIs.

Conceptual interface:

```
send(message) -> provider_message_id / accepted / error
get_status(provider_message_id) -> normalized_status
verify_webhook(request) -> verified_event
```

13.2 Provider Examples

Initial adapters may include:

- Brevo for email;
- Twilio for SMS.

These are implementation choices, not domain entities.

13.3 Message State

EventFlow should distinguish at least:

- Draft
- Queued
- Processing
- Provider Accepted
- Delivered (when provider confirms)
- Failed
- Bounced
- Cancelled
- Unknown

"Sent" should not collapse all provider states into a single Boolean once the communications module matures.

14. Background Processing

14.1 Need

Tier B-C operations may exceed safe synchronous request duration for:

- bulk emails/SMS;
- imports;
- exports;
- reporting;
- provider-status reconciliation.

14.2 Proposed Adapter

Background execution should be hidden behind a job interface so the implementation can begin with a WordPress-compatible scheduler and later evolve.

Candidate implementations may include WordPress cron or Action Scheduler, subject to an ADR and deployment constraints.

Business logic must not depend directly on one scheduler implementation.

15. Import Architecture

Recommended pipeline:

```
Upload
-> Parse
-> Normalize
-> Validate
-> Duplicate/Conflict Detection
-> Staging Result
-> Organizer Review
-> Apply
-> Audit / Summary
```

For small imports, some stages may execute in one request. The logical stages should still remain separable for Tier C growth.

16. Check-in Architecture

Reception lookup should query EventFlow's local database directly.

The search path should be optimized around:

- event ID;
- attendee normalized name;
- invitation primary name;
- optional phone/email suffix where authorized;
- check-in state;
- table assignment.

Check-in writes should be small, transactional, auditable operations.

17. Caching Strategy

Caching is optional for correctness and introduced only where measured need exists.

Appropriate cache candidates:

- event configuration;
- dashboard aggregate counts;
- immutable template renders;
- lookup lists.

The cache must never become the authoritative source for invitations, attendees, assignments, or check-ins.

18. Observability

EventFlow should progressively expose:

- application error logs;
- background job status;
- communication failure counts;
- import summaries;
- migration version;
- integration health;
- audit logs;
- future metrics.

Logs should use structured context such as event ID, operation ID, and actor ID without unnecessarily including personal data.

19. Security Architecture

Baseline controls:

1. WordPress capability checks for administrative operations.
2. Nonces for state-changing browser-admin actions where applicable.
3. Validation/normalization of all external input.
4. Output escaping by context.
5. Prepared database queries.
6. High-entropy invitation tokens.
7. No secrets in Git.

8. Provider webhook verification.
9. Rate-limit/throttle strategy for public endpoints.
10. Audit logging for sensitive changes.
11. Restricted access to exports containing personal data.
12. Explicit upload validation for invitation artwork or guest-book media.

Detailed controls move into EF-DOC-008.

20. Performance and Capacity Architecture

20.1 Tier A-B

For up to approximately 750 attendees:

- synchronous CRUD is acceptable for ordinary operations;
- admin grids are paginated;
- properly indexed custom tables are expected to be sufficient;
- background sending is preferred for bulk communications.

20.2 Tier C

For approximately 750-5,000 attendees:

- imports/exports should be job-based;
- bulk communications require batching;
- aggregates should avoid full-table scans on every dashboard load;
- search fields require appropriate indexes/normalized search columns;
- list endpoints must paginate;
- provider webhooks and jobs should be idempotent;
- operational logs must remain bounded/retained appropriately.

20.3 Future Tier D

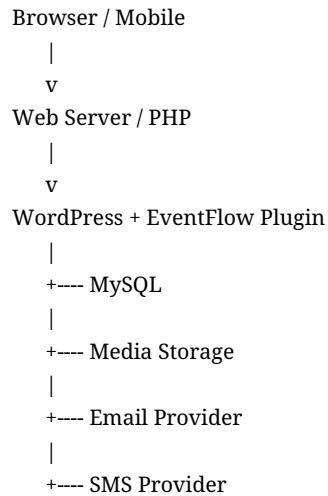
Multi-event/SaaS scale may require:

- dedicated queue infrastructure;
- stronger cache layer;
- object storage;
- separate worker processes;
- tenant isolation;
- dedicated API/application services.

Tier D is not part of the v1.x implementation commitment.

21. Deployment Architecture

21.1 v1.x Reference Deployment



21.2 Deployment Rules

- production code is versioned in Git;
- environment secrets are external to Git;
- schema migrations run through controlled EventFlow migration logic;
- backup exists before material schema upgrades;
- tagged releases identify recoverable milestones;
- production and development configuration are separated.

22. Proposed Source Structure

```
src/  
  EventFlow.php  
  Domain/  
    Event/  
    Invitation/  
    Attendee/  
    Communication/  
    Seating/  
    CheckIn/  
    Audit/  
  Application/  
    Service/  
    DTO/  
    Command/  
    Query/  
  Infrastructure/  
    Persistence/  
    WordPress/  
    Email/  
    Sms/  
    Queue/  
    Logging/  
    Presentation/
```

Admin/
Public/
Reception/
Rest/
Migration/
Support/
assets/
css/
js/
templates/
tests/
Unit/
Integration/
Acceptance/

The exact PHP namespace and folder structure will be finalized with the Developer Guide and implementation spike.

23. Migration from Current Reference Plugin

The current production reference data must not be discarded.

Proposed migration stages:

1. Inventory current table/schema and plugin version.
2. Back up database.
3. Create EventFlow Event record for Lui @ 60.
4. Create normalized Invitation records from current guest rows.
5. Create primary Attendee records.
6. Convert submitted companion JSON into companion Attendee records.
7. Preserve original tokens where secure and compatible, or rotate with controlled link regeneration.
8. Preserve submission timestamps and relevant state.
9. Validate record totals and invitation capacity totals.
10. Switch read paths to new repositories.
11. Retain a rollback window before removing legacy schema access.

A separate migration plan shall be produced after EF-DOC-005.

24. Requirement Traceability

25. Architecture Decision Candidates

The following decisions are **Proposed** and should become ADRs before implementation:

- **ADR-013:** Modular monolith architecture for EventFlow v1.x.
- **ADR-014:** Custom relational tables for core EventFlow domain entities.
- **ADR-015:** Repository interfaces between application/domain services and WordPress \$wpdb.
- **ADR-016:** WordPress REST API as the structured HTTP interface for admin/public async interactions.
- **ADR-017:** Background-job abstraction with WordPress-compatible initial adapter.
- **ADR-018:** Server-rendered guest portal with progressive enhancement for v1.x.
- **ADR-019:** Token-at-rest strategy and token rotation/revocation model.
- **ADR-020:** Controlled migration from Lui60 reference schema to Event/Invitation/Attendee schema.

These candidates are not yet Accepted.

26. Risks and Mitigations

27. Open Architecture Questions

1. Exact custom-table schema and foreign-key strategy in WordPress-hosted MySQL.
2. Whether physical foreign keys are used or integrity is enforced at application/migration level.
3. Queue adapter selection for the first communications release.
4. Exact REST authentication model for admin asynchronous requests.
5. Token hashing/lookup/rotation implementation.
6. Whether event media remains in WordPress Media Library or gains a storage abstraction immediately.
7. Audit-log retention policy.
8. Search strategy for attendee names at Tier C.
9. Formal backup/restore operational procedure.
10. Long-term boundary between WordPress plugin and future standalone/SaaS EventFlow.

These questions are intentionally visible and shall not be answered implicitly inside unrelated code.

28. Architecture Review Gate

EF-DOC-004 v0.1 is ready for review when the Product Owner confirms:

- the modular-monolith direction is appropriate for v1.x;
- WordPress remains the initial host/platform rather than the permanent product identity;
- custom EventFlow tables are acceptable in principle;
- guest access remains token-based and accountless for v1.x;
- future Tier D/SaaS complexity remains deferred;
- migration of existing Lui @ 60 data is mandatory and non-destructive.

After review, accepted decisions will be formalized as ADRs and the document will advance toward v1.0.

Appendix A - Target Data Ownership

Business Fact	Authoritative Owner
Event name/date/venue	Event
Invitation capacity	Invitation
Secure invitation token	Invitation
Primary/companion person identity	Attendee
Table capacity	Table
Seat/table assignment	Seating Assignment
Delivery attempt/status	Communication / Message Attempt
Arrival state	Check-in
Integration credential	Secure platform configuration
Material action history	Audit Log

Appendix B - Proposed ADR Candidates

ADR	Proposed Decision
ADR-013	Modular monolith for EventFlow v1.x.
ADR-014	Custom relational tables for EventFlow domain entities.
ADR-015	Repository interfaces between services and WordPress persistence.
ADR-016	WordPress REST API for structured HTTP interactions.
ADR-017	Background-job abstraction with WordPress-compatible initial adapter.
ADR-018	Server-rendered guest portal with progressive enhancement.
ADR-019	Token-at-rest hashing/rotation/revocation strategy.
ADR-020	Controlled migration from reference schema to normalized EventFlow schema.

Appendix C - Primary Risks

Risk	Impact	Primary Mitigation
WordPress coupling leaks into domain logic	High	Keep WP-specific code in infrastructure/presentation adapters.

Plugin grows without internal boundaries	High	Enforce module/service/repository boundaries and tests.
Bulk-send request timeouts	High	Background jobs, batching, idempotency.
Migration damages reference-event data	High	Backup, staged migration, reconciliation, rollback window.
Public token abuse	Medium/High	High entropy, throttling, token lifecycle controls.
Tier C admin queries become slow	Medium	Pagination, indexes, bounded aggregates, background reports.
Provider outage affects event operations	Medium	Keep core event-day operations local and provider-independent.
Premature SaaS complexity	Medium	Keep Tier D explicitly deferred.